Project 01: Linux Ops Toolkit

Real-World DevOps Projects | Part 1 of 20

Field	Details
Author	Rohan Aviral
Ticket	OPS-101: staging disk filled overnight, unnoticed for six hours
Environment	Ubuntu 24.04 container with a 200 MB /var/log, on Docker Desktop
Tools	Bash, df, du, truncate, find, gzip, cron, ss, useradd, OpenSSH
Repository	https://github.com/Aviral-Rohan/devops-project-01-linux-ops-toolkit

1. The Problem

The BrewCart staging server ran out of disk space overnight. The API stopped saving orders, and support heard about it from customers before the team did. The task: find the cause, fix it by hand, then build scripts so it cannot happen quietly again.

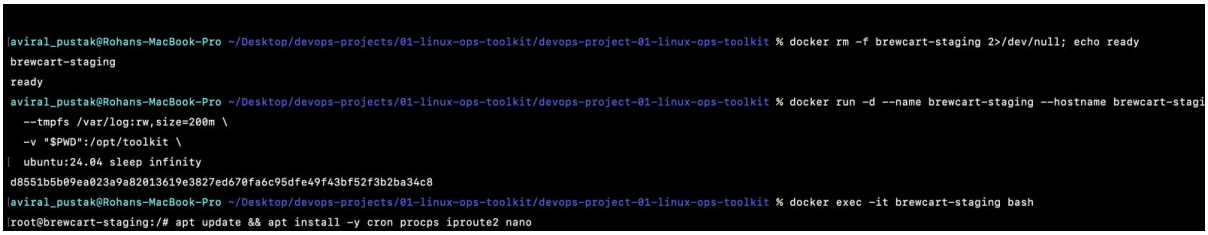
What this project delivers

Deliverable	Purpose
health_check.sh	One-command server report that exits with code 1 when a disk crosses a threshold
log_cleanup.sh	Compresses logs after 7 days, deletes after 30, with a --dry-run mode
onboard_user.sh	Creates a user and installs their SSH key with correct permissions
cron/toolkit.crontab	Runs the health check every 5 minutes and the clean-up nightly
runbooks/disk-full.md	Step-by-step response for the next disk-full incident

2. Setting Up the Staging Server

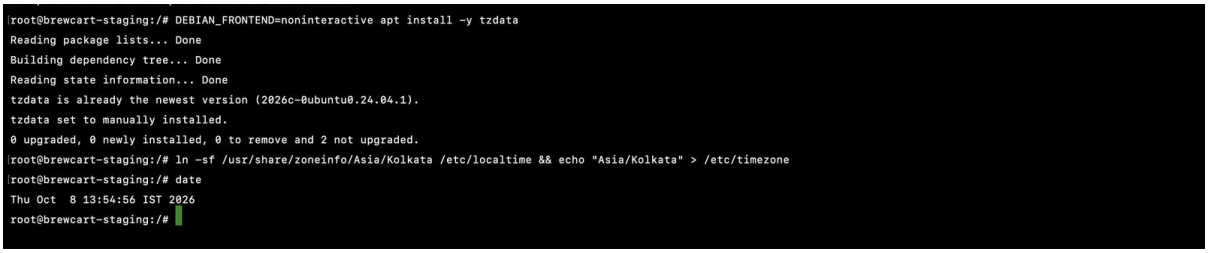
Instead of a cloud server, an Ubuntu container plays the part of staging. Its /var/log is a separate 200 MB disk, so it can fill up in seconds, and the project folder is mounted inside at /opt/toolkit: scripts are written on the Mac and run on the server.

```
docker run -d --name brewcart-staging --hostname brewcart-staging \
  --tmpfs /var/log:rw,size=200m -v "$PWD":/opt/toolkit ubuntu:24.04 sleep
infinity
docker exec -it brewcart-staging bash
apt update && apt install -y cron procps iproute2 nano
```



Screenshot 1: Container started and base tools installing

I also set the server clock to Indian Standard Time so log timestamps match local time. Containers have no systemd, so timedatectl is not available; the zone is set by linking /etc/localtime.



Screenshot 2: Time zone set with /etc/localtime; date now shows IST

3. Reproducing and Fixing the Incident

A script from the developer recreates the incident: nine dated API logs, plus a debug log that writes until the disk is full.

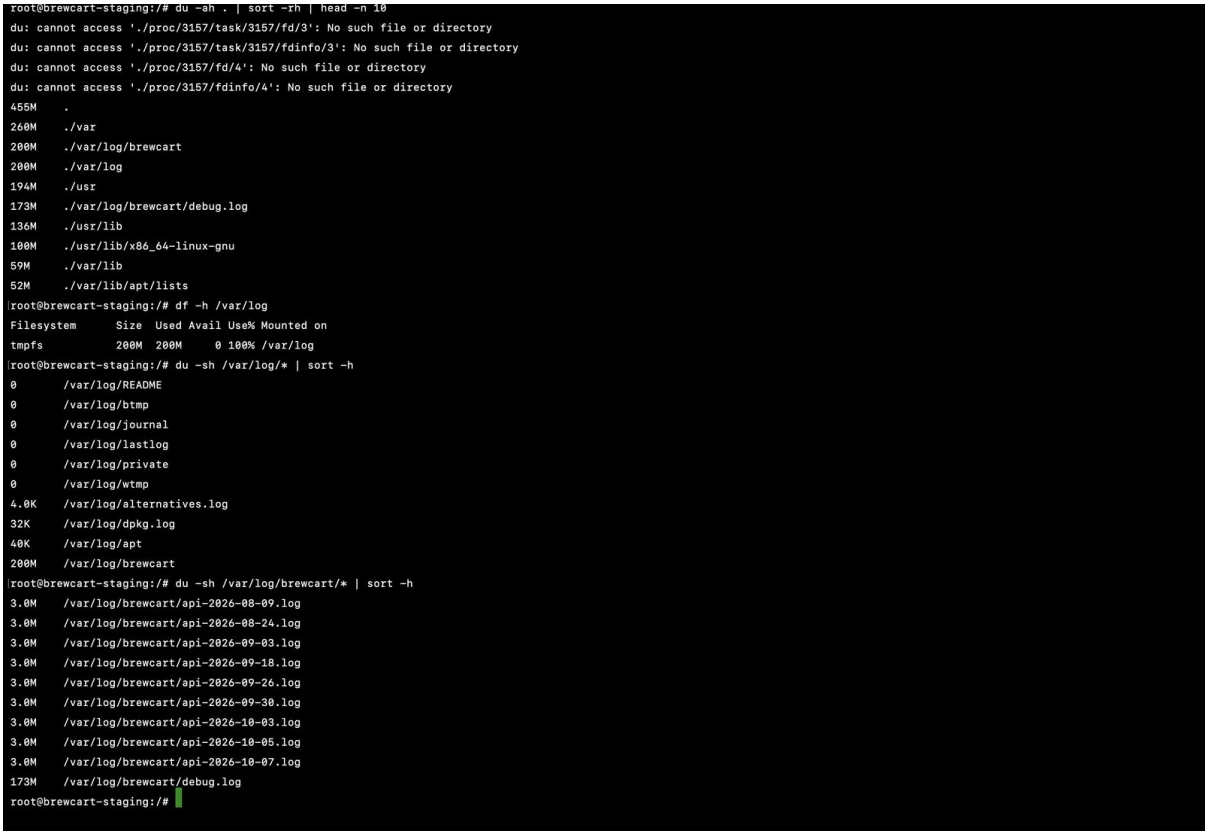
Which disk is full?



Screenshot 3: df shows /var/log at 100%

Which folder and which file?

du with sort -h lists sizes smallest to largest, so the culprit ends up at the bottom. Working down one level at a time leads straight to debug.log at 173 MB, while each normal daily log is 3 MB.



Screenshot 4: du narrows it down: /var/log/brewcart, then debug.log

Freeing the space safely

The file was emptied with truncate rather than deleted. If a process still has a file open, rm removes the name but the space is held until that process closes it, so df keeps showing a full disk. truncate -s 0 frees the space immediately.

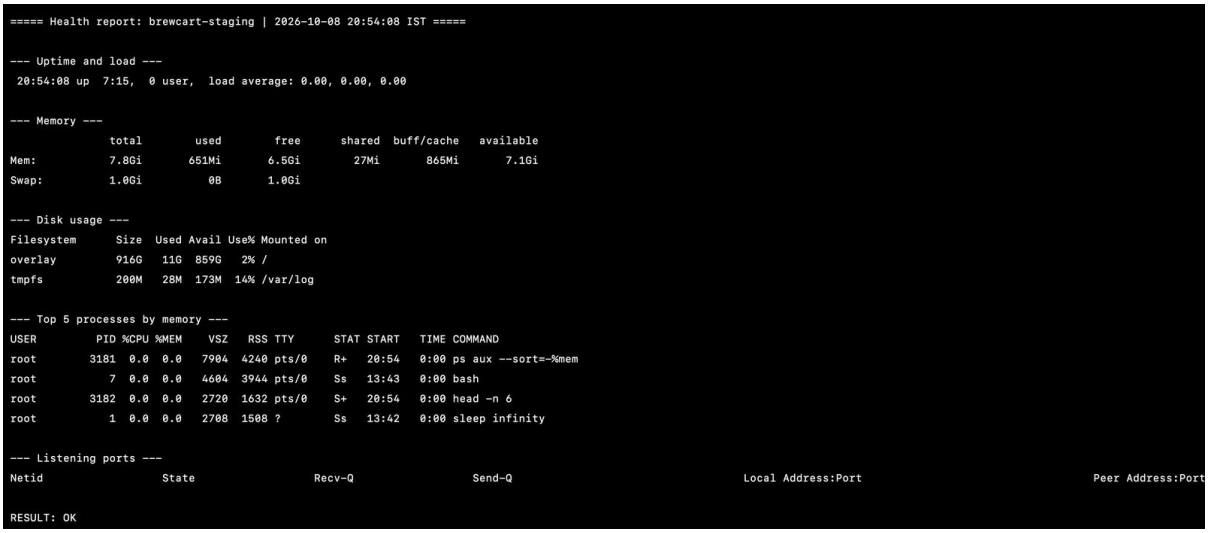


Screenshot 5: After truncate: /var/log drops from 100% to 14%

4. health_check.sh

One command that reports uptime and load, memory, disk usage, the top five processes by memory and listening ports, and exits with code 1 if any monitored disk is above a threshold (80% by default, or the first argument).

```
for mount in / /var/log; do
  used=$(df --output=pcent "$mount" | tail -1 | tr -dc '0-9')
  if [ "$used" -ge "$THRESHOLD" ]; then
    echo "ALERT: $mount is ${used}% full (threshold ${THRESHOLD}%)"
    STATUS=1
  fi
done
exit "$STATUS"
```

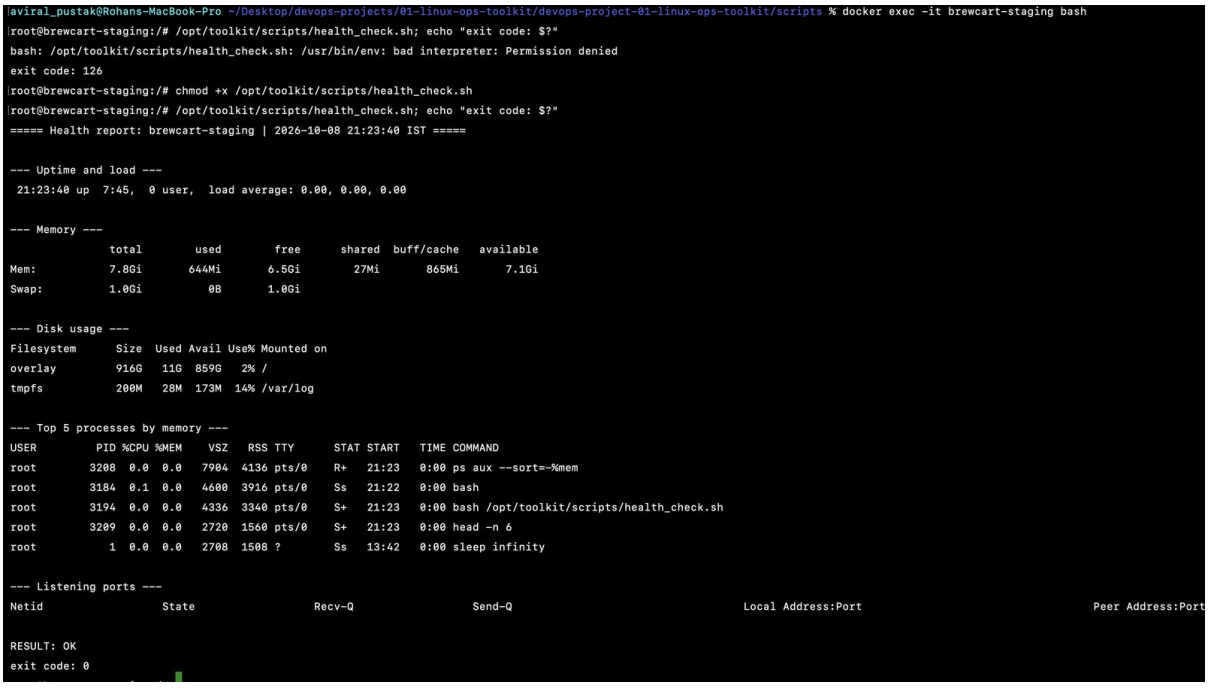


Screenshot 6: Healthy server: full report ending in RESULT: OK

Exit codes

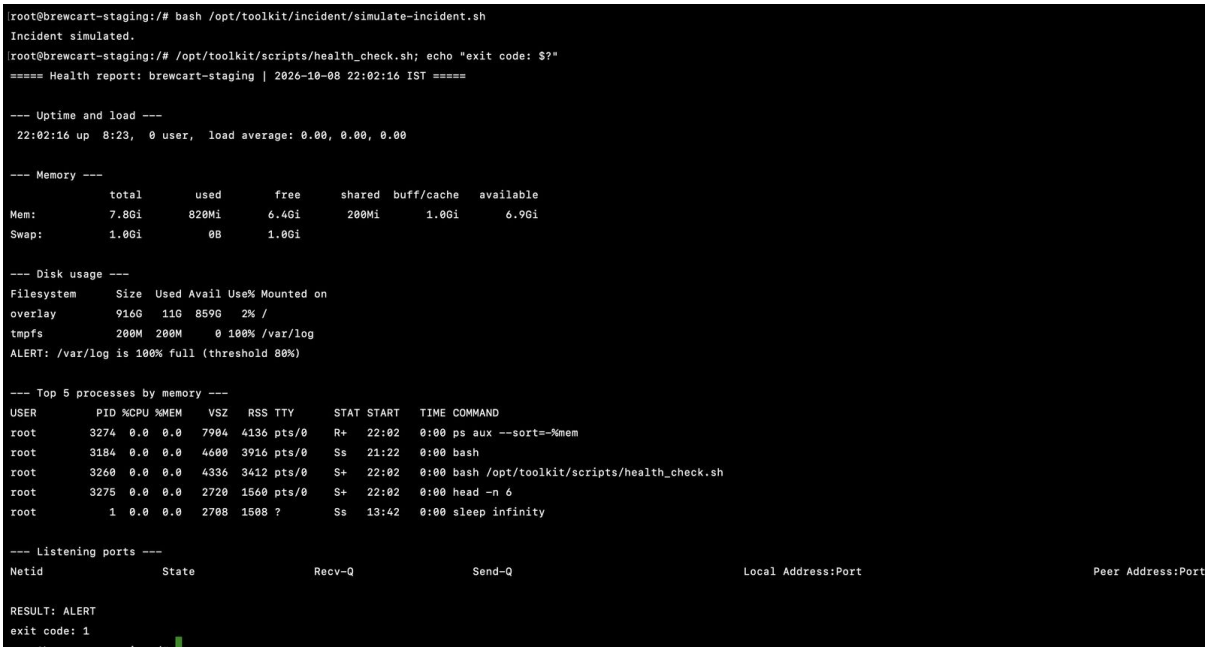
The first run failed with 'bad interpreter: Permission denied' and exit code 126, which means the file exists but is not executable. chmod +x fixed it, and the next run returned 0.

Exit code	Meaning
0	Success
1	General failure (the script's ALERT)
126	Found, but not executable
127	Command not found

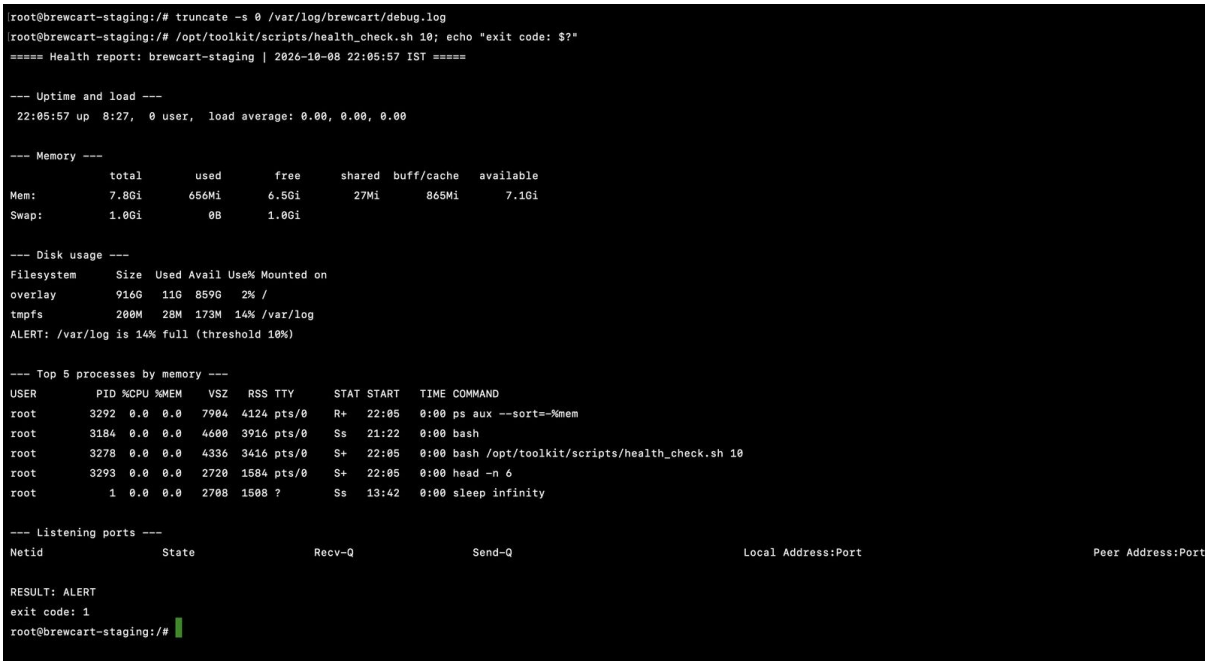


Screenshot 7: Exit code 126 before chmod +x, then RESULT: OK with exit code 0

Catching the incident



Screenshot 8: Incident re-created: ALERT /var/log is 100% full, exit code 1



Screenshot 9: Custom threshold of 10%: alerts at 14% usage, proving the argument works

5. log_cleanup.sh

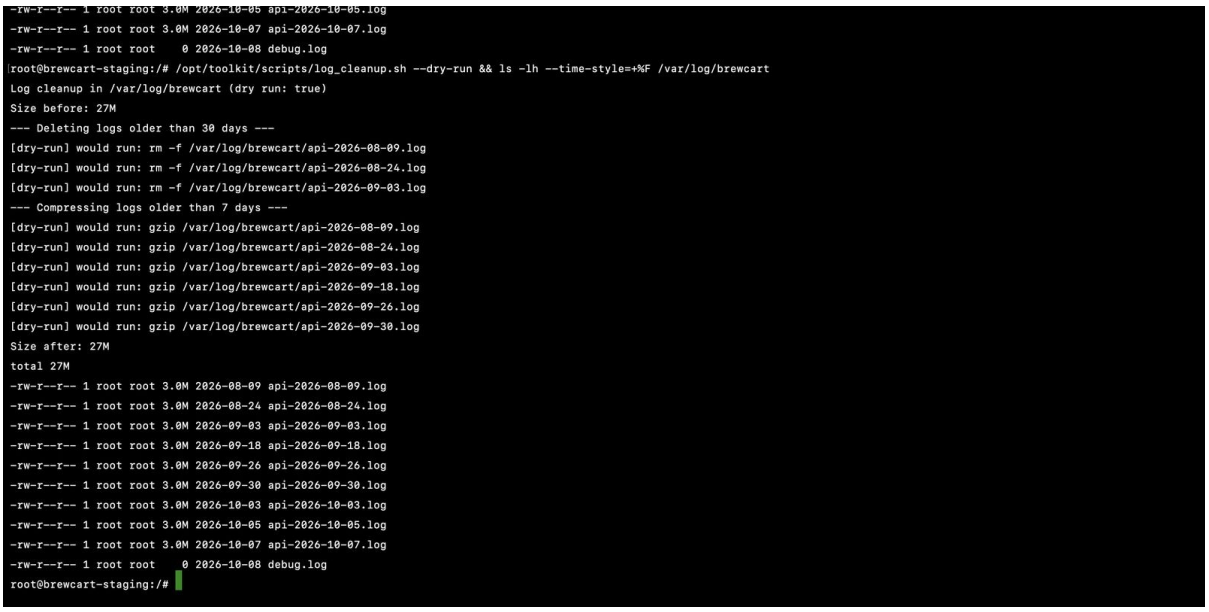
Log age	Action
0 to 7 days	Kept
8 to 30 days	Compressed with gzip
Over 30 days	Deleted

Every destructive command goes through one run() function. With --dry-run it only prints what it would do; otherwise it prints and executes. The script uses set -euo pipefail, so it stops at the first error.

```
run() {
  if [ "$DRY_RUN" = true ]; then echo "[dry-run] would run: $*"
  else echo "[run] $*"; "$@"; fi
}
find "$LOG_DIR" -type f -name '*.log' -mtime +"$COMPRESS_AFTER" \
     ! -mtime +"$DELETE_AFTER" -print0 |
  while IFS= read -r -d '' f; do run gzip "$f"; done
```



Screenshot 10: Before clean-up: nine API logs going back 60 days



Screenshot 11: Dry run: lists what would happen; the listing afterwards is unchanged



Screenshot 12: Real run: 3 deleted, 3 compressed, 27 MB down to 9.1 MB



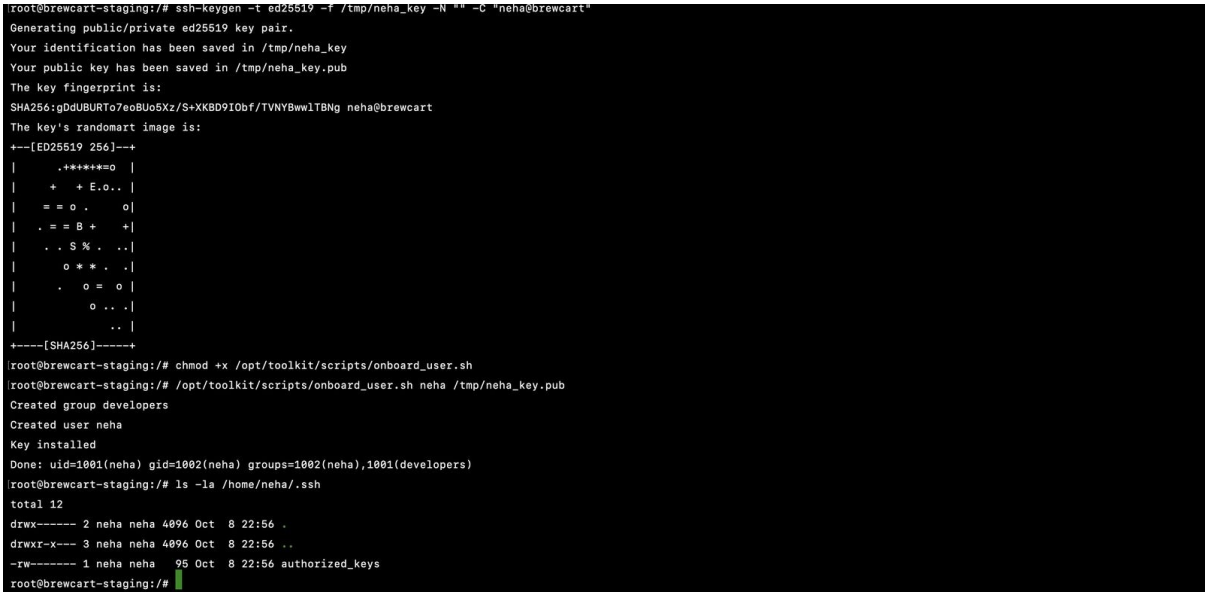
Screenshot 13: Second run does nothing: the script is idempotent

Review feedback

The first dry run said it would compress 6 files, but the real run compressed 3: the dry run counted files the delete step would remove. A dry run that differs from the real run cannot be trusted, so the compress step now skips anything older than the delete age (! -mtime +30).

6. onboard_user.sh

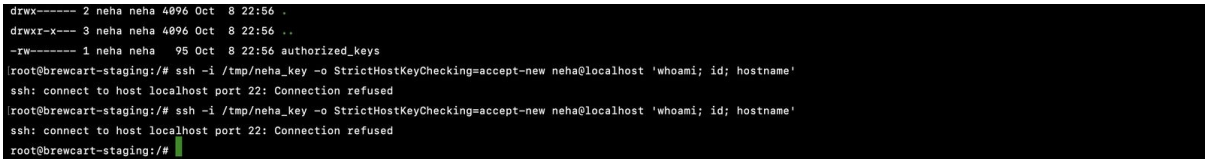
New team members get SSH-key access, never passwords. The script creates the group and user if missing, appends the public key without duplicates, and enforces the permissions sshd requires: 700 on .ssh and 600 on authorized_keys. With looser permissions sshd silently rejects the key.



Screenshot 14: Key generated (simulating the developer's laptop), user created, permissions 700 and 600

Troubleshooting: connection refused

The first login attempt failed with 'Connection refused'. That means nothing is listening on the port. Installing openssh-server inside a container does not start the service, and ss -tlnp confirmed nothing was on port 22. Starting sshd fixed it.



Screenshot 15: Connection refused: sshd installed but not running



Screenshot 16: sshd listening on :22, then a password-less key login as neha in group developers



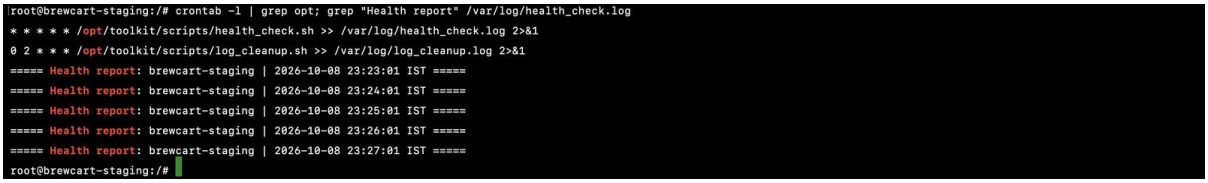
Screenshot 17: Second run: user and key already present, nothing duplicated

7. Scheduling with cron

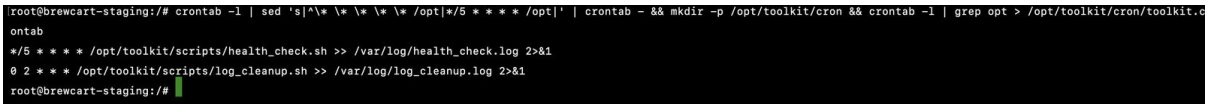
```
*/5 * * * * /opt/toolkit/scripts/health_check.sh >> /var/log/health_check.log
2>&1
0 2 * * *   /opt/toolkit/scripts/log_cleanup.sh  >> /var/log/log_cleanup.log
2>&1
```

Part	Meaning
*/5 * * * *	Every 5 minutes
0 2 * * *	Daily at 02:00
>> file 2>&1	Append output and errors to a log, so failures are not lost
Full paths	Cron runs with a minimal PATH

For the demo the health check ran every minute; the log shows one report per minute. The jobs were added without an editor, the way automation tools do it: (crontab -l; echo '...') | crontab -.



Screenshot 18: Cron writing a health report every minute



Screenshot 19: Final schedule saved to cron/toolkit.crontab in the repo

8. Key Commands

Command	Use
df -h	Disk usage per filesystem
du -xsh * | sort -h	Find what is using the space
truncate -s 0 <file>	Empty a file in place, safe while it is open
find <dir> -mtime +30	Files older than 30 days
echo $?	Exit code of the last command
ss -tlnp	Which ports are listening, and by which process
useradd -m / usermod -aG	Create a user / add to a group without

	removing others
chmod 700 ~/.ssh; chmod 600 authorized_keys	Permissions sshd requires
crontab -l / crontab -	List / install a cron schedule

9. What I Learned

- Reproduce an incident before automating the fix.
- df finds the full disk, du | sort -h finds the culprit, truncate frees space safely.
- Exit codes are how scripts talk to cron, CI and monitoring.
- Anything that deletes files needs a dry run that matches the real run exactly.
- 'Connection refused' means nothing is listening: check ss -tlnp first.

Next: Project 02, Git the Way Teams Use It.